# openvaf-r — Full Change Report

Every modification and its reason (original → current)

Ngspice + OpenVAF Enhancements project

July 2026

# Contents

# openvaf-r — Full Change Report

Every modification applied to the OpenVAF-Reloaded compiler in this project, with its reason. The baseline is the pristine OpenVAF-Reloaded source tree (as vendored in the project's `original/` snapshot); the current state is the tree committed in this repository under `OpenVAF-master-20260610/`. Each entry links the enhancement write-up in `enhancements_doc/` that carries the full engineering detail and the verifying example suite. The companion document ngspice_changes_full-report.md covers the simulator side.

> Maintenance note: this report is updated whenever an enhancement touches compiler sources. The per-enhancement index at the end tells you the last change it covers.

Scope summary. ~120 modified source files across the compiler pipeline, two new files (`hir/src/elaborate.rs` — the elaboration pass — and `hir_def/src/item_tree/diagnostics.rs`), plus three classes handled wholesale: the `test_data/` fixtures and snapshots (95 files — new test inputs and snapshots regenerated whenever a feature legitimately changed descriptor contents), the `integration_tests/*/*.osdi` reference objects (regenerated at each ABI bump), and the removal of the AGPL `va-cask` submodule. The compiler builds warning-free; every change below was regression-gated by the example-suite arsenal, the integration suite, and the VA_TEST industry-model corpus.

---

## 1. Lexer — **`openvaf/lexer/`** (**`lib.rs`**, **`cursor.rs`**)

- Based integer literals (`8'hFF`, `'sb101`): the lexer had only a commented sketch; underscores crashed it. Both the sized and unsized forms now lex, requiring a digit after the base (a silent-0 trap otherwise) (E-46).
- Don't-care digits `x/X/z/Z/?` accepted in the binary/octal/hex digit runs for `casex`/`casez` items (E-78).
- Escaped identifiers (`\my-net!`) lexed per the LRM (E-46).
- **`// comment at EOF`** hang fixed: with no trailing newline the line-comment loop spun forever on the EOF character — the only EOF-unsafe loop in the lexer (E-35).

## 2. Tokens and grammar — **`openvaf/tokens/`**, **`openvaf/parser/`**, **`veriloga.ungram`**, **`sourcegen/`**

`tokens/src/parser/generated.rs` gains the keywords and node kinds each language feature needed, threaded through `parser/src/grammar/*` (`stmts.rs`, `expressions.rs`, `items.rs`, `items/module.rs`), the grammar source of truth `syntax/veriloga.ungram`, and the `sourcegen/` generators (`ast/src.rs`, `hir_builtins.rs`, `mir_instructions.rs` — the templates the generated token/AST/builtin/instruction tables come from):

- `do … while` post-test loops (E-19);
- `paramset`/`endparamset` blocks (E-21);
- `{…}` concatenation and `{n{…}}` replication as real operators, with `'{…}` remaining the typed aggregate (E-34);
- array-literal expressions actually parsed (they were fully scaffolded but unreachable) (E-4, E-14);
- vectored (bus) net declarations and bit-selects (E-3), multi-index bit-selects for N-D arrays (E-15), and the LRM name-then-range declaration order (E-18);
- module instantiation items (E-5), `generate for`/`genvar` (E-8), and `generate if`/`generate case` with optional block labels (previously mis-parsed into a broken `GENERATE_FOR`; E-67);
- `defparam` — a reserved word with no grammar rule (E-58);
- event OR lists `@(cross(…) or timer(…))` via a new `or` keyword and looped event grammar (E-59);
- `casex`/`casez` sharing the case rule, the keyword token carrying the flavor (E-78);

- `@(initial_step)`/`@(final_step)` with analysis-phase lists (E-7, E-53);
- indirect branch assignment `V(x): V(y) == expr;` (E-2);
- the `<<<`/`>>>` arithmetic-shift tokens (plus a pre-existing lexer bug fix) (E-6);
- hierarchical parameter override targets (`#(.blk.p(4))`): the extra `.segments` are consumed so the parse stays clean instead of cascading -- elaboration then rejects the block-scoped override with a targeted diagnostic (E-87);
- hierarchical branch reference tails (`.branch(a,b)` / `.branch(<p>)`) swallowed into the path node so the enclosing item's CST stays whole for the elaboration rewrite (a keyword in path position used to shred the item, leaving the hole scanner truncated text) (E-86);
- part-selects in instance connections (`inst (out[3:2], in)`): the bit-select bracket accepts an optional `: expr`, with the colon token in the CST distinguishing a range from multi-dimensional indexing — no new node kind (E-85);
- module-level **`generate for`**/**`if`**/**`case`** without the optional **`generate`**/**`endgenerate`** keywords: `module_items` handled a `generate … endgenerate` region but had no bare `FOR_KW/IF_KW/CASE_KW` arm, so a keyword-less loop at module scope fell to error recovery — `unexpected token 'for'`, or a silent drop of the loop when a following `analog` block let recovery resync. New arms parse it into the same `GENERATE_FOR/IF/CASE` nodes the nested/`generate`-wrapped forms produce (E-96);
- panic-free direction parsing: `port_decl`/`func_arg` asserted the next token was a direction (`bump_ts`), which any non-Verilog text reaching a module-head port list turned into a compiler crash; both now emit a diagnostic with one token of forced progress (E-84);
- operator precedence corrected against LRM Table 4-2: `%` bound tighter than `*`/`/` (so `6*7%4` evaluated to 18, not 2), and xnor split from xor (E-38);
- derived-nature parents emitted the wrong node kind (`NAME_REF` where the AST wanted `Path`), leaving the entire inheritance machinery unreachable (E-39).

### 3. Preprocessor — **`openvaf/preprocessor/`** (+ **`basedb`** diagnostics)

- Ten compiler directives that previously hard-failed as undefined macros: `` `default_discipline ``, `` `celldefine ``/`` `endcelldefine ``, `` `unconnected_drive ``/`` `nounconnected_drive ``, `` `timescale ``, `` `line ``, `` `pragma ``, `` `undefineall ``, `` `default_nettype `` (E-6).
- **`` `default_transition ``** parsed and threaded to `transition()` lowering (E-47).
- Recursive macro expansion crashed the compiler (stack overflow, direct and mutual): the `MacroRecursion` diagnostic existed but was never emitted, and its report renderer was a `todo!()`. The guard pushes around the macro *body* expansion only — an entry-scoped guard false-positives on the legal `QUAD(x) = TWICE(TWICE(x))` nesting (E-65).

### 4. Syntax / AST — **`openvaf/syntax/`** (**`ast.rs`**, **`generated/nodes.rs`**, **`expr_ext.rs`**, **`node_ext.rs`**, **`name.rs`**, **`validation.rs`**, **`error.rs`**)

- Typed AST nodes for every new construct above (generated from the ungrammar).
- Literal decoding (`expr_ext.rs`): based-int parsing with size/sign handling (E-46) and the mask-aware variant returning `(value, x_mask, z_mask)` for casex/casez (E-78); a compiler crash on large bare-integer-shaped literals fixed by falling back to a float constant (E-4).
- String unescaping rewritten as a single pass: the sequential `str::replace` chain corrupted overlapping escapes (`\\n`) and octal `\ddd` was missing; all consumers route through one function (E-48).
- **`Name::resolve`** ate the last character of escaped identifiers (the committed `std.va` snapshot had baked the bug in as `logi`) (E-46).

5. Elaboration — **`openvaf/hir/src/elaborate.rs`** (new file)

The compile-time flattening pass: every stage downstream of the parser is architected around exactly one flat module per artifact, so hierarchy is resolved by recursively inlining instantiated modules — alpha-renamed per instance, ports bound to the caller's nets, parameters bound to overrides — into an ordinary flat module *before* the rest of the pipeline runs. Built for module instantiation (E-5) and grown into the home of every structural feature:

- `generate for`/`genvar` unrolling (E-8), nested loops, anonymous blocks, `generate if`/`case` with elaboration- time constant folding, and the genvar-substitution fix (genvars in ordinary expressions were routed through identifier renaming, which re-escaped `1e3*(i+1)` into a broken escaped identifier; they now become literal-value holes) (E-67);
- implicit nets from undeclared instance connections, discipline derived from the connected port, prefixed per instance so no cross-instance shorts (E-41);
- hierarchical names `u1.u2.node` and `$root` via an instance-chain map and a hole scanner (E-49);
- **`defparam`** resolved through the same chain rewrite, with the LRM-mandated precedence over instance `#()` overrides (E-58);
- net concatenation in port connections expanded bit-by-bit onto vectored ports (E-59);
- bus slicing onto instance arrays and bus ports (E-5); paramset twin-module rendering (E-21); net initializers preserved through re-rendering (E-45); escaped identifiers re-rendered correctly (E-46);
- undefined instantiation targets are a hard error — they used to be silently dropped from the rendered output, turning a typo'd module name into an invisible open circuit; discipline-named misparses (`electrical out[0:2];` parses as an instantiation) and paramsets dropped over unresolvable targets get tailored messages (E-84);
- name-then-range net/port declarations (`input in[0:2]`, `electrical out[0:2]`, LRM 3.6/3.7): a textual pre-pass rewrites the `<head> <name>[range]` form to the range-then-name form (Enhancement-3), reusing all bus/port machinery; the `(`-after-range instance rule keeps instance arrays untouched (E-89). Extended to multi-name lists (`input a[0:1], b[0:3], c;`), split into one range-then-name declaration per name with per-name widths; a multi-dimensional name is left untouched (unsupported in both orders) (E-91);
- parameter-dependent declaration widths (`electrical [0:N-1] out;`, `real w[0:N-1];`): a textual pre-pass folds a declaration range whose bounds reference a parameter to a literal range, using the module's constant-integer parameter defaults (a fixpoint resolves one default through another; `from`/`exclude` constraints are skipped). Only declaration ranges (with a `:`) are folded, not bit-selects; the shared constant-integer evaluator gained an optional parameter map (empty map = the E-88 literal-only behavior). The width is fixed at the default -- a structural parameter, since OSDI has one node count per module (E-67/88 decision) (E-91). A parameter that shaped a width is then frozen to a **`localparam`** (`freeze_width_parameters`): a multi-parameter declaration is split so only the structural names freeze, and a range constraint is dropped from a frozen name. This keeps structure and behaviour consistent under a netlist override -- without it, overriding a width parameter that also bounds a runtime loop left the frozen array size behind, a silent out-of-bounds (E-92);
- the legacy **`generate <id> (start, end [, incr])`** statement (obsolete Verilog-A 1.0 analog-block loop-unroll, LRM Annex C.4) unrolled by a textual pre-pass: the index substitutes to a literal per iteration (with bit-select bracket folding, since a bus bit-select needs a literal index), constant bounds only -- a parameter bound cannot shape the unroll (E-67 scope decision) (E-88);
- **`` `__FILE__ ``**/**`` `__LINE__ ``** expanded by a textual pre-pass run before the other passes (preprocessor tokens are (kind, span) pairs into existing source and cannot carry synthesized literals): `` `__FILE__ `` becomes the root file's basename (machine-portable provenance, the E-58 rule), `` `__LINE__ `` the exact 1-based line; string/comment occurrences are skipped and replacements are inline so later line numbers stay true (E-85);
- part-select actuals sliced onto bus ports in `bind_port` (`base[msb:lsb]`, constant bounds; positional and named forms; width-1 slices onto scalar ports degrade to the bit-select) with the same

ascending bit-order convention as full-bus slicing (E-85);
- block-scoped parameter override rejected with a targeted message (a named instance override `#(.blk.p(4))` naming a hierarchical target is collected and bailed like the unknown-module errors; block-scoped parameters are local to their block and take their value from their initializer, which may reference the module's parameters) (E-87);
- hierarchical branch probes (E-86): the top module's absolute chain map rides into every inlined child so SIBLING bodies resolve `V(top.a1.b)/$root...` references; unnamed-branch forms expand to the flattened node pair; port-branch probes `I(inst.branch(<p>))` read a 0V ammeter synthesized at flattening (which also fixes child-declared `branch (<p>)` port branches, broken after inlining); hierarchy-bound monitor modules are omitted from standalone output; `ground`/net-type declarations in inlined children keep their keyword;
- **`$port_connected`** resolved at flattening time to a literal `(1)/(0)` per instance — after inlining, an open port is just a synthesized local net, so the builtin used to fail validation in exactly the unconnected case it exists to detect; top-level modules keep the native OSDI connected-flag path (E-84).

## 6. Definition layer — **`openvaf/hir_def/`** (+ **`openvaf/hir/`** mirrors)

`body.rs`/`body/lower.rs`/`body/pretty.rs`, `expr.rs`, `item_tree*`, `nameres*`, `builtin.rs`, `data.rs`, `types.rs`, `db.rs`, plus the new `item_tree/diagnostics.rs` and the `hir` façade (`body.rs`, `lib.rs`, `db.rs`, `diagnostics.rs`):

- statement/expression collection for every new construct: `do…while` (E-19), `repeat` and `disable` (E-9), event OR lists (E-59), concat/ replication (E-34), array aggregates and whole-array assignment (E-14), `CaseKind` + per-item don't-care masks with a stray-literal list (E-78);
- N-D array machinery: per-dimension size lists, multi-index bit-selects, per-element parameter expansion (E-15); declaration initializers on (N-D) arrays via a `Var::array_index` leaf split (E-43);
- **`localparam`** made genuinely non-overridable (it behaved exactly like `parameter`) with derived localparams still tracking inputs (E-9); paramset lowering as twin modules whose bound params are localparams with override expressions (E-21);
- name resolution: `electrical ground gnd;` ordering (E-9), nature-attribute access (`net.potential.abstol` — the third "scaffolded-but-unwired resolver boundary" found) (E-45), derived-nature resolution (E-39);
- **`builtin.rs`** (the system-function registry): the noise-table pair (E-9), the full `$random`/`$dist_*`/`$rdist_*` family (E-10), file I/O and string functions (E-11), the last fallback group (`$simprobe`, aliases, plusargs) — after which `is_unsupported()` is empty (E-12), `$simparam$str` (E-25), `$realtime` (E-59), `$table_model` (E-16);
- new diagnostics file: wrong-arity calls produced invalid HIR and crashed downstream — now a proper diagnostic (functions and parameters both) (E-43);
- multiple `analog` blocks collect into `entry_stmts` in source order — the as-if-concatenated LRM semantics, by construction (E-60);
- bus-port node ordering (`item_tree/lower.rs`): a non-ANSI header bus port (`module m(in, y); input [0:2] in;`) had its first bit merged into the header placeholder but the remaining bits appended after later ports, scrambling OSDI terminal order whenever the bus was not the last port. A pre-scan of body port widths now expands the bus into contiguous bits in header order at placeholder-creation time (E-90).

## 7. Types and validation — **`openvaf/hir_ty/`**

`inference.rs` (+ `inference/fmt_parser.rs`), `builtin.rs` + `builtin/generated.rs` (signature tables), `lower.rs`, `types.rs`, `validation.rs`, `validation/body.rs`, `diagnostics.rs`:

- signature tables — a recurring defect source: `ANALYSIS` made variadic (E-30); `$table_model`

given shape-synthesized varargs signatures for any dimension (a generic-varargs fallthrough *truncated* multi-arity signature lists — varargs builtins with signature lists need their own inference arm) (E-40); `TRANSITION`'s table was one argument short (3-argument calls crashed, 4-argument worked by accident) (E-47); `transition()` input typed Real, a `$dist` typo fixed (E-49); `SIMPARAM_STR` returned Real instead of String (E-25);

- format-string inference (`fmt_parser.rs`): terminates on every conversion character and reports it, so `%5d`/`%-8s`/`%08x` type their arguments correctly — previously only real conversions accepted flags/width (E-71);
- array inference: element-wise array `case`, array-literal function arguments (which previously bound nothing and silently returned 0), integer arrays typed Real, output-literal args accepted silently — all fixed (E-33); whole-array function arguments/outputs/returns (E-18, E-20, E-23); untyped function args default to Real instead of an ICE masquerading as an initializer bug (E-43);
- **`Type::base_type`** infinite loop on any array-type check (it looped on `self` instead of the cursor) — hung the compiler on simple type mismatches (E-14);
- string handling: ternary over strings (SELECT + string binary ops appended to the operator tables) (E-37);
- validation: loop bodies get their own context so the analog-operator restriction reports "loops" (LRM 4.5.1), not "conditions" (E-70); call-graph cycles among analog functions are clean errors naming the cycle instead of a recursive-inliner stack overflow (E-59); casex/casez restrictions (stray don't-care literals, `x` digits under `casez`, non-integer discriminants) (E-78); `domain discrete` with natures rejected per LRM 3.6.2.2 (E-50); parameter defaults exempt from their own ranges (the CMC "feature disabled" idiom — stock rejected diode_cmc, BSIM-CMG, PSP-HV and the HiSIM family at setup) while given values stay fully validated (E-56);
- part-selects outside port connections diagnosed (the E-78 stray-list pattern: body lowering collects them, validation reports a dedicated error naming the supported connection form) (E-85);
- contributions to port branches diagnosed (`ContributeToPortFlow`, with the declaration site labeled): `I(pb) <+ …` on a `branch (<p>) pb;` slipped through the write path unvalidated and panicked during lowering (E-84);
- contributions to an all-**`ground`** branch diagnosed (`ContributeToGround`): `V(gnd) <+ …` / `V(gnd, gnd) <+ …` reduce both endpoints to the fixed 0 reference (no unknown) and hit an `unreachable!()` in `lower_contribute_unnamed_branch` — an ICE. The write path now checks the branch's `is_gnd` nodes and reports a clean error; a real node-to-ground branch and a `V(gnd)` probe are untouched (E-97).

## 8. Lowering — `openvaf/hir_lower/`

`expr.rs`, `stmt.rs`, `ctx.rs`, `callbacks.rs`, `parameters.rs`, `state.rs`, `fmt.rs`, `lib.rs`:

- analog operators: `laplace_nd/np/zd/zp` via exact state-space realization (E-4) with complex pole/zero pairs per the LRM's (re, im) convention (E-31); `zi_*` via bilinear transform (E-6); `slew`/`transition` as a saturating tracking loop (E-6) with the LRM negative `max_neg_rate` convention honored (the sign defect made `slew` ignore its input and ramp away) (E-61) and a DC-singularity clamp via an `EnableIntegration` identity (E-47); `idtmod`'s modulo wrap moved off the reactive residual (it diverged) and an argument- index bug fixed (E-27); `idt` initial conditions survive into transient (E-28); the `idt` assert/reset forms with smooth reset dynamics (E-52); `limexp` kept stateless as a documented decision (E-13);
- **`$table_model`**: 1-D piecewise-linear lowered to differentiable MIR so autodiff yields the Jacobian for free (E-16); N-D multilinear as recursive 1-D (E-17, E-40); natural cubic splines with the precomputed moment matrix (E-22);
- events: `cross`/`above`/`timer` with persistent previous-value slots (E-8); OR lists folded with a select-based bool-or (a raw `ior` ICEs const-eval) (E-59); final-step gating and phase-list AND-ing (E-53);
- variable persistence: genuine per-instance `hidden_state` storage behind a two-pass MIR build

(E-7), typed slots so integer state stopped crashing instruction selection (E-32);

- callbacks (`callbacks.rs`, `fmt.rs`): the `$display` family with the full `[flags][width][.precision]` surface and `%h/%b/%r` translation (E-71); file I/O and string formatting through a generalized `PrintDst` sink (E-11); the deterministic seed-and-salt RNG lowering (E-10); `$simparam$str` (E-25); simulation-control return flags (E-55); `$discontinuity` (E-24);
- operator fixes: unary ~ emitted arithmetic negate; the constant folder sign-extended >> where the runtime was unsigned — const and runtime disagreed (E-37);
- whole-array coercion (`expr.rs`, `stmt.rs`): inference records a coercion of a whole array as a cast on the array *expression*, but `lower_array_elems_impl` decomposes the array and lowers each element itself, so `lower_expr`'s `needs_cast()` never saw it — the cast was silently dead, and every new array-consuming context re-inherited the same crash (an integer element reaching a float MIR op, which const-eval has no case for: *"invalid operation fdiv Int(1) Float(..)"*). Patched per-call-site four times — the `case` discriminant (E-33), `laplace_*/zi_*` integer-literal then integer array-variable coefficients, and an integer `case` item against a real discriminant — before being fixed at the chokepoint, which now honours the recorded cast for every consumer (E-214). The explicit `coerce_real` flag remains for consumers whose element type is fixed by the language rather than by an inferred cast (a coefficient vector is real per LRM 9.19, and inference records no cast for it);
- masked casex/casez comparisons (two `iands` ahead of the existing integer equality) (E-78); array function-argument writeback (E-20) and array returns via a function-named var-array (E-23);
- named port branches (`branch (<p>) pb;` — LRM 3.7.2): a flow probe of a PortFlow-kind branch routes through the same `CurrentKind::Port` param as a direct `I(<p>)`, so E-29's defining equation covers both spellings; probing one used to hit `unreachable!()` (E-84);
- literal **`if`** conditions lower only the taken branch — a dead analog operator (`transition()` under `if ((0))`, the exact shape the `$port_connected` rewrite produces) survived const-folding as a detached-but-interned op whose state setup read optimized-away values and aborted codegen (E-84).

## 9. MIR core and optimization — **`openvaf/mir*`**

`mir/` (`instructions.rs` + `instructions/generated.rs`, `dfg.rs` with `dfg/phis.rs` and `dfg/uses.rs`, `dominators.rs`, `flowgraph.rs` with `flowgraph/transversal.rs`, `layout.rs`, `cursor.rs`, `lib.rs`, `builder/generated.rs`), `mir_opt/` (`simplify_cfg.rs`, `simplify.rs`, `const_eval.rs`, `global_value_numbering.rs`), `mir_autodiff/` (`builder.rs`, `lib.rs`, `live_derivatives.rs`), `mir_llvm/` (`builder.rs`, `intrinsics.rs`), `mir_build/`, `mir_interpret/`:

- three pre-existing general CFG bugs, all found via event-counter verification: a dangling-reference bug in unreachable-block removal, a multi-exit post-dominance bug in the dominator-tree builder, and a block-merge bug that could corrupt which block the DAE builder treated as the exit (E-8);
- the post-dominator sink-root pathology that let dead-code elimination delete op-dependent `$fatal` calls (side-effecting callbacks under op-dependent control now stay in eval; control dependence computed by arm-reachability) (E-55);
- `split_tainted.rs` tolerates layout-detached branch instructions (a branch whose condition const-folded has no CFG effect to taint; it used to unwrap and panic) (E-84);
- const-eval agreement with runtime semantics for shifts and the bitwise-not fix (E-37);
- autodiff: noise operators process before any `ddt` (a shared ddt evaluated between two noise operators silently dropped the second source), and the reactive coupling of a `ddt`-shaped noise wave is registered so small-signal pruning keeps it (E-54);
- **`llvm.fabs.f64`** registered in the LLVM intrinsics table — it never had been, and the signed noise-power convention needed it (E-42);
- new opcodes/instructions supporting the operator work (barrier/ integration-identity forms; MIR deliberately has no `fabs`, so lowering composes it) (E-47, E-52).

## 10. DAE construction — **openvaf/sim_back/**

`dae.rs` + `dae/builder.rs`, `topology*` (incl. `lineralize.rs`, `small_signal_network.rs`), `noise.rs`, `context.rs`, `lib.rs`:

- implicit equations for indirect branch assignment (one new unknown
  - residual per statement) (E-2) and the absdelay synthetic two-node form (E-1) — with equation indices kept stable by mapping dead/collapsed unknowns to zero-contribution placeholders instead of renumbering;
- port-flow probes **`I(<port>)`**: a TODO stub returning 0 became a synthesized DAE unknown mirroring the node's KCL residual (E-29);
- probe-only branches: probing a never-contributed branch read 0 and an open circuit — a 0 V-source pass materializes them, enabling ideal ammeters and flow-only signal-flow disciplines (E-36);
- noise factors (`noise.rs`): equation-path noise was silently lost (never attached to the DAE); factors generalize to `re + jω·im` with no extra matrix unknowns (E-54); same-named source grouping metadata (E-42);
- `idt` reset-mode charge dynamics and conditional step bounding (E-52).
- voltage-source branches feeding internal nodes were open circuits at DC: the small-signal (noise/`ac_stim`) pruner keyed node registration on the branch's LIVE voltage unknown, so a pure `V(a, f) <+ expr` (V(a,f) never read) registered nothing and the node's conduction silently moved to the AC-only residual; any voltage-capable branch now disqualifies its nodes (E-86);
- probed **`V(x,y) <+ 0`** branches were node-collapsed away, making `I(branch)` read zero; hint pairs whose branch current is a DAE unknown are suppressed (the unprobed collapse idiom is untouched) and `NodeCollapse::hint` tolerates suppressed pairs; pinned by the permanent `vsrc_internal_node` snapshot test. Behavior change: a collapsible branch whose current the model references (MVSG_CMC's access resistances carry noise on `flow(d,drc)`) stays a real 0V source — electrically identical, two extra matrix rows (E-86).

## 11. OSDI code generation — **openvaf/osdi/**

`lib.rs`, `metadata.rs` + `metadata/osdi_0_4.rs`, `inst_data.rs`, `eval.rs`, `load.rs`, `setup.rs`, `compilation_unit.rs`, `ndatable.rs`, `stdlib.c`, `build.rs`, reference headers:

- descriptor emission for every ABI evolution (see the ngspice report's ABI table): the absdelay/last_crossing info arrays (E-1, E-6), `OsdiNode.nodeset` (E-45), the `ac_stim` source array + `load_ac_stim` (E-51), stride-2 signed noise pairs in `load_noise` (E-54) — the version tuple lives in `osdi/src/lib.rs`;
- **`metadata.rs`**: the `PARA_FLAG_FIXED` parameter-descriptor flag (a free bit of the `flags` field, additive — no layout/ABI change) set on every `localparam`, so the simulator can warn when a netlist tries to set a non-settable parameter (a `localparam`, or a structural width parameter frozen by Enhancement-92) (E-93);
- **`eval.rs`**: final-step flag gating (E-53); simulation-control return flags (E-55); the `ac_stim` large-signal value (was an `unreachable!()` panic) (E-26);
- **`inst_data.rs`**: absdelay slot layout (E-1); hidden-state slots typed by the variable (`ty_f64` was hardcoded — integer state fed f64 loads into integer MIR ops and aborted instruction selection) (E-32);
- **`compilation_unit.rs`** (print codegen): the `%b` segfault — the binary-formatted string was remembered for `free()` but never pushed to the `snprintf` argument list, so the matching `%s` consumed a garbage pointer (E-71); the print-callback machinery with its shadowed-`fun` and volatile-table IPO traps (E-11);

- **`stdlib.c`**: the file-descriptor table behind `$fopen`/`$fdisplay`/... (E-11); the splitmix64 `osdi_rng_*` runtime (E-10); a `simparam_str` loop/return bug (E-25);
- **`ndatable.rs`**: `ddt`/`idt` natures resolved through `resolve_nature_index` instead of panicking on derived natures (E-39);
- **`setup.rs`**: range validation with default-exemption (E-56); a 64-line abandoned unsafe experiment (`VoidAbortCallback`) deleted in the warning cleanup (E-66);
- **`build.rs`**: the copied-target stdlib compilation trap fixed (E-10).

## 12. Driver, libraries, and infrastructure

- `openvaf-driver/src/main.rs`: hard errors now exit non-zero — the error arm printed the failure but fell through to a success exit, so every elaboration failure looked like a successful compile to shell scripts (a quirk first noticed during E-58's work) (E-84);
- `openvaf-driver/src/crash_report.rs`, `linker/src/lib.rs`, `lib/bforest/` (`map.rs`, `pool.rs`, `set.rs`), `lib/typed_indexmap/` (`set.rs`): the zero-warning cleanup (44 → 0 across 13 crates: lifetime annotations, dead code, nightly cfgs, an unused LLVM-prefix probe) (E-66);
- `basedb/` (`ast_id_map.rs`, `lints.rs`, `lib.rs`, `diagnostics/preprocessor_error.rs`, `diagnostics/syntax_error.rs`): AST-id plumbing for elaboration-created items (the `AstId::from_erased` placeholder used by array returns; E-23), preprocessor/syntax diagnostic rendering for the new errors;
- `.llvm-version` pins the LLVM 18 toolchain expectation; the `hir` and `preprocessor` `Cargo.toml`s carry the dependency additions their new code needed.

## 13. The test suite — **`openvaf/openvaf/tests/`**, **`lib/mini_harness/`**, **`test_data/`**, **`integration_tests/`**

The fork's own integration suite over real compact models had never run in this project: its OSDI loader was frozen at ABI 0.4, its optional VACASK legs panicked on a never-initialized submodule, and the tests hide behind `RUN_DEV_TESTS=1`. Fixed test-side only (E-68):

- `tests/load/osdi_0_4.rs` + `load/mod.rs`: loader structs synced to ABI 0.7; `mock_sim/mod.rs`: stride-2 noise convention; `mini_harness`: missing directories skip with a note instead of panicking;
- VACASK removed entirely (`.gitmodules`, the submodule, 34 harness legs and their stale snapshots): AGPL-3.0 cannot be vendored here;
- `test_data/` (95 files as a class): new fixtures for new features and snapshots regenerated whenever descriptor contents legitimately changed — every regeneration reviewed (the diffs are the project's own features: `flow(<port>)` unknowns, probe-only branches, implicit-equation nodes); `integration_tests/*/*.osdi` regenerated at each ABI bump.

---

## 14. Per-enhancement index (compiler-touching enhancements)

E-1 *sim_back, osdi*

`absdelay()` synthetic-node DAE + descriptor arrays

E-2 *parser→hir_lower, sim_back*

indirect branch assignment (implicit equations)

E-3 *parser, hir_def*

vectored (bus) nets with bit-selects

E-4 *hir_lower, syntax, parser*

`laplace_*` state-space + array-literal parsing

E-5 *elaborate.rs (new), parser*

module instantiation via compile-time flattening

E-6 *preprocessor, parser, hir_lower, osdi*

directives, shifts, slew/transition, zi_*, last_crossing

E-7 *hir_lower (state), osdi*

`@(initial_step)` gating + variable persistence

E-8 *parser, elaborate, hir_lower, mir/mir_opt*

generate for + events; three general CFG bug fixes

E-9 *hir_def, hir_lower, sim_back*

noise tables; localparam/ground/strings; repeat/disable

E-10 *hir_lower, osdi stdlib*

`$random`/`$dist_*` deterministic RNG

E-11 *hir_lower, osdi*

file I/O + string functions (PrintDst)

E-12 *hir_def, hir_lower*

last unsupported builtins as LRM fallbacks

E-13 *hir_lower*

`limexp` kept stateless (documented decision)

E-14 *hir_def, hir_ty, hir_lower*

array literals/params/dynamic indexing; base_type hang

E-15 *hir_def, hir_ty, hir_lower*

N-D arrays

E-16/17/22/40 *hir_ty, hir_lower*

`$table_model`: 1-D, N-D multilinear, cubic spline, varargs sigs

E-18/20/23/33 *hir_ty, hir_lower, basedb*

arrays in analog functions: in/out/return/literals + array case

E-19 *tokens→hir_lower*

`do … while`

E-21/44 *parser, elaborate, hir_def, sim_back*

paramset + hidden system parameters

E-24/55 *hir_lower, mir, osdi*

`$discontinuity`; `$finish`/`$stop`/`$fatal` honored

E-25 *hir_ty, osdi stdlib*

`$simparam$str`

E-26/51 *osdi, sim_back*

`ac_stim`: crash fix, then full AC-RHS injection

E-27/28/52 *hir_lower, sim_back*

the `idt` family: modulo, IC, assert/reset

E-29/36 *sim_back*

port-flow probes; probe-only branches

E-30 *hir_ty, hir_lower*

variadic `analysis()`

E-31 *hir_lower*

complex poles/zeros in root forms

E-32 *osdi inst_data*

integer persistent state (crash fix)

E-34 *tokens→hir_lower*

`{…}` concat + `{n{…}}` replication

E-35 *lexer*

EOF comment hang

E-37/38 *hir_lower, mir_opt, parser*

operator + precedence audits (~, >> fold, `%` level)

E-39 *parser, hir_ty, osdi*

derived natures unwired at the node-kind boundary

E-41/49/58/59 *elaborate*

implicit nets; hierarchical names; defparam; port concat + OR lists

E-42/54 *sim_back noise, mir_autodiff, mir_llvm, osdi*

correlated + node-free complex noise factors

E-43 *hir_def (+diagnostics), hir_ty*

initializers completed; arity diagnostics

E-45 *osdi, hir_def, elaborate*

nodesets + nature-attribute access (ABI 0.5)

E-46/48 *lexer, syntax*

escaped ids, based literals, single-pass unescaper

E-47 *preprocessor, hir_ty, hir_lower*

`` `default_transition `` + transition fixes

E-50 *hir_ty validation*

domain-binding validation

E-53 *hir_lower, osdi eval*

`@(final_step)` + phase lists

E-56 *hir_ty, osdi setup*

parameter defaults exempt from ranges

E-59 *tokens→hir_lower, hir_ty*

`$realtime`; OR lists; recursion diagnostics

E-61 *hir_lower*

`slew` sign fix (operator-args audit)

E-65 *preprocessor*

macro-recursion guard

E-66 *13 crates*

zero-warning cleanup (44 → 0)

E-67 *tokens, parser, syntax, elaborate*

generate audit: genvar fix, nesting, if/case

E-68 *tests, mini_harness*

integration suite enabled; VACASK removed

E-70 *hir_ty validation*

loop-context diagnostics

E-71 *hir_ty fmt, hir_lower fmt, osdi*

display-format surface + `%b` segfault

E-78 *lexer→hir_lower, hir_ty*

`casex`/`casez` don't-care masks

E-84 *parser, elaborate, hir_ty, hir_lower, mir_opt, driver*

LRM example sweep: 6 defect fixes (port-branch panic, parser robustness, silent undefined modules, `$port_connected` on open ports, dead-op codegen, exit codes)

E-85 *parser, elaborate, hir_def, hir_ty*

`` `__FILE__ ``/ `` `__LINE__ `` + connection part-selects (the last two sweep findings)

E-86 *parser, elaborate, sim_back*

hierarchical branch probes + 2 DAE fixes (V-source-to-internal open circuit, collapse-of-probed-branch)

E-87 *parser, elaborate*

block-scoped parameters (validated) + clean diagnostic for the illegal `#(.blk.p())` override

E-88 *elaborate*

legacy `generate <id> (start,end)` analog-block loop-unroll (textual pre-pass)

E-89 *elaborate*

name-then-range net/port decls (textual normalize) + Annex E SPICE-primitives example library

E-90 *hir_def (item_tree)*

multi-bit input bus port bit reads: pre-scan body widths so a non-last bus port's bits stay contiguous in header/terminal order

E-91 *elaborate*

multi-name name-then-range declarations + parameter-dependent declaration widths (folded from the parameter default)

E-92 *elaborate*

freeze structural (width) parameters to `localparam` (split multi-parameter decls) so a netlist override cannot desync the frozen width from behaviour

E-93 *osdi (metadata)*

flag `localparam` OSDI parameters non-settable (`PARA_FLAG_FIXED`, additive) so ngspice can warn on a netlist override (ngspice side too)

E-96 *parser (module grammar)*

parse a module-level `generate for/if/case` without the optional `generate/endgenerate` keywords

E-97 *hir_ty (validation)*

clean diagnostic (was an ICE) for a contribution to an all-`ground` branch (`V(gnd) <+ …`)

E-101 *hir_ty (builtin), mir_opt / mir_interpret / mir_llvm*

`$clog2`: accept one argument (`INT_MATH_1`, was a 2-arg signature) and compute `ceil(log2 n)` = `bit_width(n-1)` in all three backends (was `floor(log2 n)+1`, wrong on exact powers of two)

E-102 *parser (items), syntax (ungrammar + ast), hir_def (item_tree)*

name-then-range array-valued parameters (`parameter real c[0:2]`); `parameter()` accepts post-name `[range]`, `lower_param` resolves dims per name

E-103 *mir_llvm (intrinsics)*

register the `llvm.ceil.f64` intrinsic (was missing; `ceil()` of a non-constant argument crashed codegen -- `floor` was registered)

E-104 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr)*

add `$rtoi` (real->int, truncate toward zero via `ficast((x<0)?ceil:floor)`) and `$itor` (int->real) conversion builtins

E-105 *hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)*

`$sscanf`/`$fscanf` honour the format conversion base -- parse the format string, dispatch integer fields to base-specific scanners (`%h`/`%x` hex, `%o` octal, `%b` binary)

E-106 *hir_ty (types, inference), hir (signatures), hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)*

string relational comparison (</<=/>/>=) via a lexicographic `osdi_strcmp` callback (`RELATIONAL_COMPARISON` adds the STR signature; `a<op>b` lowers to `strcmp(a,b)<op>0`)

E-107 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)*

add `$fgetc(fd)` single-character read as a `FileOp::Getc` -> `osdi_fgetc` (completes the file I/O family)

E-108 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)*

add `$ungetc(c, fd)` one-character pushback as a 2-arg `FileOp::Ungetc` -> `osdi_ungetc` ( companion to `$fgetc`)

E-109 *hir_lower (callbacks), osdi (load)*

correct `noise_table` (piecewise-linear in `f`) and `noise_table_log` (log-log, `P=10^lerp(log10 p, log10 f)`) interpolation to LRM 4.6.4.3/4.6.4.4 -- both were nonconformant (lin-log, and a mis-keyed log-freq input)

E-147 *hir_ty (validation/body)*

fix exponential-time compilation of nested **`?:`**. Found by a robustness campaign (117 production CMC models + ~50 adversarial inputs + 4000 mutation-fuzz iterations). The body validator's `validate_expr` ends by recursing into children via `walk_child_exprs`; arms that validate their own operands `return` first (`Call`, `Path`), but the `Select` (ternary) arm did NOT -- it validated `cond/then_val/else_val` via `validate_condition` and then fell through, validating `then_val/else_val` a SECOND time, so a chain of N nested `?:` was validated 2^N times. Depth ~30 ( reachable via macros) hung the compiler; `sample`-profiling confirmed an unbounded `validate_expr↔validate_condition` recursion with doubling call count. Fix: add `return;` to the `Select` arm (the operands are already fully validated). O(2^N)→O(N): depth 40 hang→0.09 s, depth 2000→1.6 s. Behaviour-preserving: all 117 models give the IDENTICAL verdict before/ after (0 flips, head-to-head), BSIM4 (12.6k lines) still ~2.3 s, and hir_ty/hir/hir_lower/sim_back tests pass with no snapshot changes. Campaign also confirmed 0 panics/segfaults across 4000 fuzz iterations; remaining lower-severity findings (parser stack overflow on ~4k-32k-deep nesting, `include` self-recursion, huge array dimension) are documented follow-ups. Verified 7/7 (nested_ cond_examples: depth 20/40/80/160 all <0.2 s, linear growth, correct piecewise value in ngspice)

E-148 *parser (parser, grammar/expressions), preprocessor (diagnostics, processor), basedb (diagnostics/ preprocessor_error), hir_def (item_tree, item_tree/diagnostics, item_tree/lower), hir (elaborate)*

compiler hardening: closes the three lower-severity robustness findings from E-147 (pathological input → crash/hang) by turning each into a clean bounded diagnostic. (1) Parser expression-depth limit: a shared `Parser::expr_depth` counter, incremented on each `atom_expr` (recursion) and per operator in the `expr_bp` loop (operator-chain / tree depth), bounds total expression-tree depth at 1000 and recovers to the next statement boundary when exceeded — so `----…x` / `x+1+1+…` / `((…))` / `sin(sin(…))` / `1?…:0` no longer overflow the recursive-descent parser OR a later recursive tree traversal (`expr_bp` split into a save/restore wrapper + `expr_bp_ inner`; `atom_expr` into an inc/check/dec wrapper + `atom_expr_inner`; reuses the existing `err_recover/unexpected_tokens_msg` recovery). (2) **`` `include ``** recursion cap: an `include_ depth` counter on the preprocessor's `Processor` caps nesting at 64, emitting a new `Prepro- cessorDiagnostic::IncludeRecursionLimit` (rendered in basedb) instead of overflowing the stack on a self-including file — mirrors the E-65 macro-recursion guard. (3) Array element cap: a shared `array_elem_count` helper (product of `|msb−lsb|+1`, capped at 2^20 ≈ 1.05M) + a new `ItemTreeDiagnostic::ArrayTooLarge`, applied at EVERY expansion site — variable arrays, parameter arrays, net/port buses, array function returns (all in `hir_def` item-tree `lower. rs`), and instance arrays in BOTH the item-tree lowering and the `hir` elaboration pass (which re-expands `dev s[0:N]()` into rendered text) — so `real x[0:100000000]` (and param/bus/ instance equivalents) degrade to a single scalar with an error instead of exhausting memory. Behaviour-preserving: all 117 production CMC models give the IDENTICAL verdict before/ after (0 flips, head-to-head), BSIM4 still ~2.3s; parser/preprocessor/hir_def/hir_ty/sim_back unit tests pass with no snapshot changes; 0 build warnings. Verified 17/17 (robustness_examples: 5 deep-expression shapes + self-include + 4 huge-array kinds all error in <1s no crash/hang with expected diagnostics; valid nested-ternary-30 / parens-100 / 100-term-sum / small arrays still compile). Fully resolves the E-147 campaign's remaining findings

E-185 *mir_autodiff (builder.rs)*

autodiff audit: hypot & atan2 derivative fixes. A deep audit compared the AC small-signal conductance dI/dV (built by the mir_autodiff automatic differentiation that feeds AC/convergence/noise/pz) of a battery of nonlinear laws I=f(V) against the ANALYTIC f'(V), and caught two builtins right in VALUE (DC correct) but wrong in DERIVATIVE -- the accidental-correctness pattern, first on the compiler side. (1) hypot(x,y): the autodiff rule computed (x'+y')/(2*hypot) -- the sqrt(x) pattern x'/(2*sqrt) misapplied to a 2-arg function -- instead of the correct (x*x'+y*y')/hypot; 28% off at V=0.7, y=0.5, only accidentally correct at x=0.5. Fixed by splitting hypot from sqrt in inst_cache (cache holds hypot(x,y), not 2*hypot) and a correct chain rule with the usual zero/one folding. (2) atan2(x,y): two bugs in the cached factors feeding the shared Pow|Atan2 chain rule (x'*c0 + y'*c1)*c2 -- the common factor c2 was (x^2+y^2) where the rule MULTIPLIES, so it needed the reciprocal 1/(x^2+y^2); and c1 (the y' factor) was +x where d/du atan2 = (x'*y - y'*x)/(x^2+y^2) SUBTRACTS, so it needed -x. Wrong magnitude AND sign. Fixed: c2 = 1/(x^2+y^2), c1 = -x. The fix is at the autodiff-rule level so it applies to resistive currents, reactive charges (verified via AC susceptance of ddt(hypot)), higher-order derivatives, and ddx. verify_vafautodiff: 9 checks (both arg orders, sqrt-form equivalence, the V=0.5 accidental-correctness point, atan2 sign + atan2(V,V)->0 cancellation, reactive susceptance) + a 15-builtin regression battery confirming the rest were already correct. Regression 150/150.

E-187 *mir_opt (simplify.rs)*

math-identity simplifier: invalid inverse-function cancellations. The algebraic simplifier (simplify_unary_op) rewrites f(g(x)) -> x for function-inverse pairs. Six of these were applied unconditionally even though the cancellation is only valid when f is a true left inverse of g over ALL of g's range -- so they returned the raw inner x, WRONG for ordinary finite inputs, corrupting the DC value itself (a more severe class than the derivative-only autodiff bugs). (1) asin(sin(x)) != x for |x|>pi/2 (asin(sin 3)=pi-3=0.1416). (2) acos(cos(x)) != x outside [0,pi] (acos(cos 4)=2pi-4). (3) atan(tan(x)) != x for |x|>pi/2 -- and this is a legitimate angle-WRAP idiom the optimizer silently defeated (atan(tan 2)=2-pi). (4) acosh(cosh(x)) != x for x<0 (cosh even -> |x|). (5,6) sqrt(x*x) and sqrt(x**2) are |x|, not x (sqrt((-3)^2)=3, returned -3). The const-fold path (eval_unary) evaluates each op numerically and was always correct, so constant spot-checks passed -- the bug lived only in the symbolic cancellation on runtime values. Fix: Asin/Acos/Atan/Acosh decline to simplify (return None); the Sqrt arm returns None (MIR has no fabs, so the sqrt stays and computes |x|). Kept the cancellations valid over the whole real line (tan(atan), ln(exp), asinh(sinh), atanh(tanh), sinh(asinh), cosh(acosh), log10(pow(10,.))). Rewrites unsound only for Inf/NaN (x-x->0, x*0->0, sin(asin(x)), exp(ln(x))) left as standard finite-math. Removed the now-unused F_TWO import. mir/mir_opt/mir_autodiff unit tests unchanged (13/7/19). New example mathident_examples/verify_mathident.py: 12 DC-value checks. Regression 151/151.

E-186 *mir_autodiff (builder.rs + lib.rs)*

autodiff audit: real-modulo (%) derivative fix. The same AC-conductance referee (E-185) caught a third builtin right in VALUE, wrong in DERIVATIVE. Verilog-A real modulo lowers to the Frem opcode; x % c = x - floor(x/c)*c is a slope-1 sawtooth in x, so d/dx(x % c) = 1 away from the wrap points -- yet Frem was grouped with the genuinely-constant opcodes (floor, ceil, clog2, integer/bitwise ops, comparisons) and forced to derivative 0 in TWO places : (1) the live-derivative gate zero_derivative() in lib.rs, which decides which SSA values even depend on -- with Frem listed, a modulo result was declared independent of the unknown so its derivative was never returned (no-edge arm). Because the gate came first the AC/Jacobian contribution of any modulo term a.dc sweep of $I = 1e-3 * (V$ clog2/integer ops correctly stay in the zero group. Applies everywhere the derivative is taken -- resistive I, reactive Q, ddx, higher orders. verify_vafautodiff extended to 14 checks (adds 5 modulo cases: two constant divisors, prefactor scaling, chain-rule d/dV (V%1)^2 = 2*(V%1), and floor/ceil staying 0), a separate 3-terminal probe confirming the variable-divisor branch, cross-checked via the ddx value path; the 19 mir_autodiff unit tests are unchanged and

pass. Regression 150/150.

E-213 *syntax (lib.rs, parsing/tree_builder.rs), lexer (lib.rs), parser (grammar/paths.rs), preprocessor ( parser.rs), examples/vafcrash_examples/**

crash hardening: four compiler panics. Fuzzing openvaf-r with malformed Verilog-A found four distinct panics: instead of printing a diagnostic the compiler aborted with "OpenVAF encountered a problem and has crashed!" (exit 101) and directed the user to file a bug report -- for what is often just a typo. All are reachable from ordinary source mistakes; the headline case is a module merely missing its `endmodule`, one of the most common Verilog-A editing errors. BUG 1 -- EOF SPAN (tree_builder.rs): a parse error at end of file built its span as TextRange::at(self.text_pos, ); text_pos is already the end of the source, so adding the previous token's length produced a span running PAST the end of the file -- 6..12 for the 6-byte input "module". Mapping it back to its file then failed assert!(range.end() <= self.range.end()) in FileSpan::with_subrange (preprocessor/sourcemap.rs), crashing the compiler WHILE TRYING TO PRINT THE ERROR ("subrange 6..12 -> 6..12 must fit into the total range 0..6"). FIX: an empty range at EOF -- exactly what the adjacent expected_at field already does. Also find_ctx_range (syntax/lib.rs) maps a position through half-open [start,end) ranges that never cover the EOF position itself (pos == last_range.end() compares Less against every range); it .expect()ed and panicked, now clamps. Triggers: missing endmodule; bare `module`; module header then EOF; unclosed `analog begin`; unterminated string. BUG 2 -- REAL LITERAL (lexer/lib.rs): e/E was consumed as an exponent marker without checking an exponent follows (eat_float_exponent()'s return, which reports whether a digit was seen, was discarded), so 1e became a Float token whose text does not parse as f64 and panicked in ast::StdRealNumber::value()'s src.parse().unwrap(). FIX: an e only joins the number when a digit (optionally signed) follows; a bare 1e lexes as 1 + identifier e and surfaces as an ordinary parse error -- the approach based_literal_body already takes for a malformed 8'squark. Valid exponents (1.5e3, 2e-3, 1E6) untouched. Triggers: 1e, 1e+, 99e, 1.5e, parameter real p=2e. BUG 3 -- PREPROCESSOR EOF (preprocessor/parser.rs): previous_range() (self.full_tokens[pos], reached while building the "expected an identifier" diagnostic) and followed_by_bracket_without_space() (self.relevant_tokens[self.pos + 1u32]; "index out of bounds: the len is 2 but the index is 2") indexed the token list directly, out of bounds one past the last token -- a bare `"define" ending the file. FIX: both use .get() with a sensible fallback, mirroring the .get().map_or() idiom current_range() already used. BUG 4 -- PATH() ASSERT (parser/grammar/paths.rs): path() opened with assert!(p.at_ts(PATH_SEGMENT_TS)), a precondition several callers do not check and plausible input violates: aliasparam x = 5;(a literal where a parameter name belongs),aliasparam x = ;, I(<1>)/I(<>)(port_flow),discipline d 1 = 2;. FIX: drop the assert -- the next line, p.expect_ts(PATH_SEGMENT_TS), already emits "expected identifier" and returns false, so the error is reported and an empty path node completed (nature's parser guarded its own call site this way in E-39; this makes the helper safe for every caller). This is the openvaf-r counterpart to E-212 (the same campaign against ngspice) and a direct continuation of E-148: E-148 hardened against pathological DEPTH (inputs too big), E-213 covers inputs that stop too early or are malformed. No change to any accepted program -- every fix is on a path that previously aborted the compiler; generated OSDI for every existing model is identical. Verify (examples/vafcrash, 25 checks): every repro yields a clean error instead of a crash; valid code unchanged (resistor; real exponents; aliasparam bound to a real parameter; define with args). NOTE: openvaf-r installs a CUSTOM panic hook printing its own message and exiting 101 rather than dying on a signal or printing the usual Rust "panicked at" -- a crash check looking only for those two (as E-148's suite does) scores these panics as ordinary errors, so the new suite treats exit 101 and the hook message as a crash. Toolchain tests unchanged (lexer 8, preprocessor 6, basedb 9, sim_back 26, hir 15, hir_lower 4). 25 checks. Regression 173/173 (new example folder).

E-214 *hir_lower (expr.rs, stmt.rs), examples/arraycast_examples/**

whole-array type coercion: a recurring crash class, fixed at its root. An integer Value reaching a float MIR op (feq/fmul/fsub/fdiv) panics mir_opt::const_eval::eval_binary, which has no (Int, Float) case -- "invalid operation fdiv Int(1) Float(..)", exit 101, "please open an issue". (Unfolded, the same defect instead reaches LLVM as `i32 = fadd .., ConstantFP:f64` and aborts with "LLVM ERROR: Cannot select" -- one bug, two faces.) FOUR INSTANCES, each patched at its own call site as found: the `case` discriminant over an integer array (E-33, element type hardcoded real); laplace_/zi_ integer-LITERAL coefficients (b77266ec); laplace_/zi_ integer ARRAY-VARIABLE coefficients (c55812d6 -- the previous fix had reasoned "a whole-array variable reference is already real by its declaration", which is false for an `integer` array); and an integer `case` ITEM against a REAL array discriminant (8d0ab057 -- the opcode is chosen from the discriminant, Feq, but the item's elements lower to i32). ROOT CAUSE: inference DOES record the coercion (expect( ) -> casts.insert(item, Array{Real})), but it lands on the array EXPRESSION, and lower_array_ elems_impl decomposes the array and lowers each element itself -- so lower_expr's needs_cast( ) never saw it and the cast was silently DEAD. Every new array-consuming context therefore re-inherited the trap. FIX: lower_array_elems_impl -- the one place every whole-array consumer passes through -- now honours a recorded cast to a real target (coerce_real |= matches!(needs_ cast(expr), Some((_, dst)) if *dst.base_type() == Type::Real)), making inference's intent effective for all consumers instead of requiring each call site to ask. Proven in isolation: with the case-site flag disabled the structural guard alone fixes every case repro. lower_case additionally keeps its own discr_op == Opcode::Feq coercion -- choosing the opcode from the discriminant makes matching operand types that function's invariant, independent of inference's bookkeeping. NO MISCOMPILE: the integer spelling of a filter is bit-identical to the '{1.0} spelling (worst AC diff exactly 0 dB over 13 points) and matches the analytic response to 3e-8 dB; an integer `case` item still selects its arm. MUTATION-TESTED: reverting the fixes makes all four repros crash (exit 101) again, so the guard is not vacuous. Also folds in two verification guards hardened during the same hunt -- vafautodiff (8->16 checks: cross-derivatives with both arguments live, the blind spot that hid E-185's hypot bug, plus a 44-point battery) and operator_examples (+14 const-vs-runtime checks: every check there used literal operands, exercising only the folder -- the side E-37's >> bug happened to be on). New examples/arraycast_examples (23 checks). Regression 174/174.

E-215 *hir_ty (builtin.rs), hir_lower (callbacks.rs, expr.rs), osdi (compilation_unit.rs, stdlib.c), examples/plusargs_examples/**

$test$plusargs / $value$plusargs productionized. E-12 had gated these as mechanism-unavailable fallbacks ($test always FALSE, $value never wrote its output). E-215 serves them through the simparam channel (ngspice publishes each command-line +name[=value] as namespaced sim-params -- see the ngspice report). COMPILER side: $test$plusargs("name") lowers to simparam( "$test$plusargs$name",0)!=0; $value$plusargs("name=%fmt",var) builds the key from the compile-time literal and reads by TARGET TYPE -- $valnum for int/real (ficast for an int target), and a NEW non-fatal simparam_str_opt for a string target. simparam_str (E-25) FATALS on an unknown name, which a plusarg probe must not, so simparam_str_opt (stdlib.c + the SimParamStrOpt callback + compilation_unit wiring) returns the default instead. hir_ty: VALUE_ PLUSARGS's 2nd arg became an OUTPUT target -- three signatures Var(Integer)/Var(Real)/Var( String) (mirroring $random's seed / $fgets's buffer) instead of the read-only Val(String) that made extraction impossible; TEST_PLUSARGS's arg is now Literal(String). DESIGN NOTE: reading the value through the op-dependent simparam channels DELIBERATELY avoids the $sscanf scanner, whose scanf_begin/scan_* thread a hidden module-global cursor with no explicit data dependency -- the setup/eval partitioner can hoist a bias-independent scan_* into instance setup while its scanf_begin stays in eval, so the scan reads a NULL cursor and segfaults (observed, then designed out). $value$plusargs matches only the name=value form per the LRM. No OSDI ABI change; old .osdi unaffected. Verify (examples/plusargs, 12 checks both solvers). Regression 175/175 (new example folder).

E-219 *preprocessor (grammar.rs), basedb (diagnostics/sink.rs), examples/robustness_examples/**

preprocessor macro-argument hang + diagnostic-flood cap. Re-running the openvaf-r robustness campaign against the shipped binary (see the robustness report) found a FIFTH hang path the E-147/E-148 guards did not cover, at a ~2% mutation-fuzz rate. FINDING A -- INFINITE LOOP: a backtick token followed by `( (name() is a macro call, and the preprocessor collects its parenthesised argument list token by token in parse_macro_call -> parse_macro_token. The SAME non-advancing pattern exists in the define PARAMETER list loop (parse_define). Neither collector had a forward-progress guarantee: when a non-Macro compiler directive (`include, ifdef, endif, undef, ...`) appeared inside the argument list, parse_macro_token pushed an UnexpectedToken error and RETURNED WITHOUT CONSUMING the token, so the caller's collection loop re-examined the same directive token forever -- an unbounded diagnostics-vector growth that pins the CPU (a `sample` of a hung compile named the loop exactly: process_token -> parse_macro_call -> parse_macro_token -> RawVec::grow_one -> realloc). It is trivially reached: injecting `( anywhere near an include (or any macro token) splits the directive so the rest of the file is scanned as a macro argument. (define did NOT trigger it -- compiler_directive() has no define arm, so it falls through to Macro and is consumed as a bogus macro name, which happens to advance; only the RECOGNISED directives hit the non-advancing branch.) A stray delimiter in a define parameter list that is neither an identifier, ) nor , (e.g. a / or " in a corrupted define) is likewise consumed by none of the loop's expect/eat calls, so parse_define spins on it. FIX (grammar.rs): guarantee forward progress -- the stray-directive branch of parse_macro_token consumes the offending token after reporting it (p.bump); and BOTH collection loops (macro-call args and define params) gained a backstop that records the source offset and bails with a clean UnexpectedEof if an iteration consumes nothing, so no non-advancing token, present or future, can spin them. No valid model puts a directive inside a macro call's parentheses, so valid input is unaffected. FINDING B -- DIAGNOSTIC FLOOD: with A fixed, deeply nested NON-macro garbage (e.g. sin( x3000 in a declaration) no longer looped but still took ~40s -- it produces thousands of parse errors and codespan_reporting builds a full source-annotated report for EVERY one; rendering ~3000 diagnostics (each extracting and laying out its surrounding source) is the entire cost (it persists with stderr redirected to /dev/null -- it is the BUILDING, not the writing). A bounded-but-40-second rejection is still a DoS vector. FIX (sink.rs): cap the number of diagnostics the console sink RENDERS at 128; beyond that only the counters advance and a one-line 'further diagnostics suppressed' note prints. summary() still reports the true error total and the exit code is unchanged -- the standard 'too many errors' behaviour of rustc/clang, which does not affect any input with <=128 diagnostics (every real model and every ordinary mistake). RESULT: name( + stray include: infinite -> clean error <0.01s; real model + injected (': >90s -> <0.05s; sin( x3000 in a declaration: ~40s -> <1s; a few real errors: unchanged, all shown. Continuation of E-148 (E-148 hardened the parser expr-depth / include recursion / array expansion; E-219 covers the two paths E-148 did not touch -- the preprocessor macro-arg collector and the diagnostic sink). Verify: examples/robustness_examples gains eight argument-collection cases -- five macro-call (m( then include/ifdef/endif/undef, and one with 4000 leading '(') and three define parameter-list (M(a,/,b), M(a"b), M(a;b)) -- plus a valid macro-call-with-nested-parens regression (26/26); the production corpus (VA_TEST/compile_all.py) still compiles 92/92 standalone models to the identical verdict; the campaign fuzzer's ~2% hang rate (3000 iterations) drops to 0 hangs, 0 crashes. Two files (openvaf/preprocessor/src/grammar.rs, openvaf/basedb/src/diagnostics/sink.rs). No change to any valid program's output beyond the >128-diagnostic suppression note.

E-220 *parser (parser.rs), syntax (lib.rs), preprocessor (grammar.rs, sourcemap.rs), basedb (diagnostics.rs), hir_ty (diagnostics.rs, inference.rs, validation.rs, validation/body.rs), examples/vafcrash2_examples/**

crash hardening round 2: ten compiler panics -> clean errors. A second robustness pass, continuing E-213 and E-219. Re-running the mutation fuzzer against the shipped compiler with DIVERSE compact-model seeds (BSIM/HICUM/PSP/HiSIM/VBIC/MEXTRAM/EKV) and new mutation strategies (keyword, attribute (* *), and bracket injection alongside byte/truncate/delimiter/deep-nest) found ~5% of mutated inputs still CRASHED the compiler (exit 101, 'OpenVAF encountered a problem and has crashed!') rather than reporting an error. Every one is the E-213 pattern -- a panic/assert/unwrap/index a valid model never reaches but malformed input does (all caught by the E-213 panic hook, so never memory-unsafe; the bug is the crash UX + missing diagnostic). TEN root causes, all fixed: (1) parser/parser.rs -- the parser can spin in error recovery; a step counter assert!(steps<=10M,'seems stuck') panicked. It now signals EOF at the limit so parsing winds down and reports its errors. (2) basedb/diagnostics.rs -- a diagnostic built from an empty node list hit to_unified_span_list([])=>unimplemented!(); it renders label-less, anchored to the new SourceMap::root_file(). (3) preprocessor/grammar.rs -- TextRange::new(start, end) with start>end (a macro-argument/`define`/include span at EOF) tripped a text-size assert; five sites clamp end>=start. (4) hir_ty/diagnostics.rs + validation.rs -- 28 sites did expr_map_back[e].unwrap()/stmt_map_back[s].unwrap(); a SYNTHESIZED expression/statement has no source-map-back entry, so reporting a type error on it panicked. They resolve the span through a fallback (empty range) via one expr_range helper. (5) hir_ty/validation/body.rs -- 11 sites did expr_types[arg].unwrap_node()/unwrap_branch()/unwrap_port_flow(), which unreachable!() when a builtin (e.g. $port_connected) or nature access gets a wrong-typed argument inference did not reject; they bail cleanly (match {Ty::X(id)=>id,_=>return}). (6) syntax/lib.rs -- to_ctx_span mapping a span across source contexts could yield start>end in TextRange::new; clamp. (7) preprocessor/sourcemap.rs -- FileSpan::with_subrange asserted the subrange fit its parent (E-213 fixed one trigger at its source; this makes the mapping itself total); clamp into the parent. (8) preprocessor/grammar.rs -- stripping include quotes with path[1..len-1] panicked on a malformed/unterminated string literal (a lone quote); saturating_sub + get make it total. (9) hir_ty/inference.rs -- resolving a call whose arguments match NO overload left the candidate list empty after the semantic-retain and candidates[0] indexed out of bounds; it falls back to the pre-filter set ( mirroring the existing restore after the exact-match retain). (10) hir_ty/validation/body.rs -- the builtin-validation arms index args[0..2] assuming the call has as many arguments as the builtin requires; a call with too few (e.g. $simparam(), $port_connected()) indexed out of bounds. ONE entry guard skips builtin validation when args.len() < BuiltinInfo::from(call).min_args -- inference already reports the ArgCntMismatch. Method: the fuzzer classifies OK/clean-error/CRASH/HANG; each crash was triaged from its crash log to the innermost openvaf frame, grouped, and the panic read at the source; fixes were applied one cause at a time, each verified against the recorded crash corpus, then a fresh fuzz re-checked convergence (each pass exposed the next-rarest site -- a classic fuzzing tail -- until the rate hit zero). Result: the ~390-input recorded crash corpus all clean-errors; a fresh 12000-iteration fuzz on the fixed compiler is 0 crashes / 0 hangs (down from ~5%); the 92/92 standalone production models compile to the identical verdict; parser/syntax/preprocessor/basedb/hir_ty/hir/sim_back unit suites pass. Verify (examples/vafcrash2): 19 checks with hand-crafted inputs targeting each cause, MUTATION-TESTED (reverting the fixes makes the guarded inputs crash again, so the suite is not vacuous). No OSDI/ABI change; generated .osdi for every existing model is identical. Eight files. Regression 179/179 (new example folder).

E-230 *hir_def (item_tree/lower.rs), hir_ty (validation.rs), syntax (ast/expr_ext.rs), examples/vafcrash3_examples/**

crash hardening round 3: three compiler panics -> clean errors. A third robustness pass (following E-213/E-219/E-220). The production corpus recompiled (92/92 standalone, identical verdicts) plus a fresh ~19,500-iteration mutation-fuzz over diverse compact-model seeds found THREE more distinct panic root causes, all the E-213 class (caught by the panic hook, memory-safe, but a crash instead of a diagnostic). (1) hir_def/item_tree/lower.rs -- a `begin :` sequential block with the scope colon but a MISSING/invalid name identifier has `block_scope().is_some()` yet

‘name = block_scope().and_then(

E-261 *mir_autodiff (builder.rs), examples/vafsqrtguard_examples/**

autodiff: guard the **`sqrt()`** derivative singularity. `d/dx sqrt(x) = 1/(2*sqrt(x))` is `+inf` at ngspice's default `x=0` DC initial guess; openvaf-r emitted that raw `+inf` into the Jacobian, which produced a `nan` and made the operating point fail outright -- a conductor `I=K*sqrt(V)` never found ANY DC op (dynamic/true gmin, source, and pseudo-transient stepping all died, node = `nan`). The tell was an internal inconsistency: `Pow` carried an explicit `base==0 -> derivative:=0` guard ”to ensure numerical stability” while `Sqrt` had none, so the mathematically identical `pow(V,0.5)` and `V**0.5` converged and ngspice's own behavioral B-source `sqrt(v(n))` converged, but `sqrt(V)` NaN-failed. (The `Pow` guard is itself incomplete -- a block split that only protects a bare terminal `pow`/`sqrt`; `2.0*pow(V,0.5)` fails on the unmodified compiler because the downstream multiply consumes the raw derivative.) FIX (`inst_cache`): change the `Sqrt` derivative cache from `2*sqrt(x)` to `2*sqrt(x + a)` (`a = 1e-18`) -- the exact derivative of the smoothly regularized `sqrt(x+a)`, so the emitted derivative is `x'/(2*sqrt(x+a))`. It is FINITE at `x=0` (`1/(2*sqrt(a)) ~ 5e8`, a large but bounded conductance -> small controlled Newton steps that creep out of the singularity, exactly like the B-source); EXACT for `x>0` (the nudge is INSIDE the root, so the perturbation is `~a/(2x)`, below the ULP -- no finite-bias derivative changes, higher-order derivatives and the internal `sqrt(1-x^2)` of asin/acos/asinh/acosh/atanh included); and COMPOSABLE (being a plain value it propagates through downstream operators `K*sqrt(x)`, `1/(1+sqrt(x))`, `exp(-sqrt(x))`, unlike the block-split guard). Two alternatives were rejected: a block-split `Sqrt` arm mirroring `Pow` (does not compose -- scaled `sqrt` still fails), and an additive `2*sqrt(x)+delta` (perturbs the derivative `~delta/(2*sqrt(x)) ~ 1e-9`, breaking the bit-exact higher-order asin/acos autodiff unit tests) -- the `sqrt(x+a)` form is exact to the ULP AND convergent, so NO test tolerance was loosened. `pow(x,fractional)` is unchanged (its base derivative `(y/x)*x^y` is an `inf*0` form in the shared `Pow`/`Atan2` chain rule with no clean branchless regularization; `sqrt()` is the standard spelling). Verify (examples/vafsqrtguard, both solvers): bare and strongly-scaled `K*sqrt(V)` (K=1,2,5) find their true KCL op -- not `nan` -- and match the equivalent B-source to ~1e-5; the guarded derivative `K/(2*sqrt(V))` is exact for `V>0` (~1e-6); composed `G0/(1+sqrt(V))` converges; `pow(V,0.5)` now agrees with `sqrt(V)`. openvaf-r's own autodiff suite (incl. numeric third-order asin/acos exactness at `10*eps`) and the OSDI/sim_back MIR snapshots pass unchanged in value. Two source files.

E-262 *mir_autodiff (builder.rs), examples/vafsqrtguard_examples/**

autodiff: guard the **`pow(x,y)`** base-derivative singularity (the E-261 sqrt fix, applied to pow). For `0<y<1`, `d/dx pow(x,y) = y*x^(y-1)` is `+inf` at the `x=0` DC initial guess -- the same singularity as sqrt (pow(x,0.5) IS sqrt(x)). openvaf-r builds it via the chain rule shared by Pow and Atan2: `d(pow) = (x'*cache[0] + y'*cache[1])*cache[2]` with `cache[0]=y/x`, `cache[2]=x^y`, so the base term `x'*(y/x)*x^y` is `+inf*0 = NaN` at `x=0` (the `y/x` factor is `+inf`, the `x^y` factor is `0`) -- it NaN-poisons the Jacobian and the operating point fails. Pow carried a PARTIAL guard (a block split forcing the derivative to 0 when `base==0`), which is why a BARE terminal `pow(V,0.5)` converged -- but a block split cannot compose: a downstream instruction (`2*pow(V,0.5)`, `1/(1+pow(V,0.5))`) consumes the RAW derivative computed inside the conditional block, so scaled/combined fractional pow still NaN-failed on the unmodified compiler. FIX (`inst_cache`, mirroring E-261's sqrt): cache the derivative of the a-shifted `pow(x+a, y)` (`a = 1e-18`) while the VALUE `res = x^y` is unchanged -- `cache[0] = y/(x+a)`, `cache[1] = ln(x+a)`, `cache[2] = (x+a)^y`. The shared chain rule then yields base term `x'*y*(x+a)^(y-1)` and exponent term `y'*ln(x+a)*(x+a)^y`, both FINITE at `x=0` (`y*a^(y-1)` is large but bounded -> a controlled Newton step out of the singularity, like the sqrt guard and ngspice's B-source), EXACT for `x>0` (the nudge is inside the power, perturbation `~a/x` below the ULP), and PLAIN VALUES so they compose through downstream operators. With a composing branchless guard in place the old block-split Pow guard is removed. Atan2 -- which shares the chain-rule ARM but has its own cache -- is untouched and verified unchanged; `y>=1` is a no-op (no singularity); the solution-dependent exponent term is now

finite at x=0 too (ln(x+a) not ln(0)=−inf). ln(x)/1/x are left as-is (their VALUE, not just the derivative, is +-inf at 0, so no derivative guard well-poses a model that evaluates them at 0). Verify (examples/vafsqrtguard [6]/[7], both solvers): bare and strongly-scaled K∗pow(V,Y) (K=1,2,5 at Y=0.5,0.3,0.25) find their true KCL op -- not nan -- where the unmodified compiler NaN-failed the scaled/fractional cases; the guarded derivative K∗Y∗V^(Y−1) is exact for V>0 (~1e-6). openvaf-r's autodiff suite (incl. atan2 and higher-order checks) and the OSDI/sim_back MIR snapshots pass unchanged in value; the many production models using pow (BSIM/PSP/...) are bit-unchanged for V>0. Completes the autodiff singular-derivative work (E-261 sqrt + E-262 pow). Two source files.

E-263 *sim_back (init.rs), hir_ty (inference.rs), hir (elaborate.rs), test_data/ui/ddx.log, examples/vafcrash4_examples/**

robustness fuzzing: three compiler panics -> clean errors (the 4th such pass, following E-213/E-219/E-220/E-230). Three fuzzing strategies against the committed compiler -- byte/token mutation of the whole .va corpus, grammar-aware STRUCTURED adversarial inputs, and VALID-but-pathological modules that compile through to the backend -- surfaced one distinct panic each (all the E-213 class: panic-hook-caught, memory-safe, but a crash instead of a diagnostic). (1) sim_back/init.rs -- deeply nested analog operators (I<+absdelay(ddt(ddt(absdelay(.. ))))) produced a value tagged for the instance-setup cache whose mapping in the INIT function is a Const or has no definition at all (ValueDef::Invalid); build_init_cache assumed every cached value is a computed instruction result and called unwrap_result()/unwrap_inst(), and even once survived the OSDI LLVM backend read a BuilderVal::Undef. FIX: before allocating a cache slot, handle the non-instruction cases -- substitute a Float/Int/Str/Bool CONSTANT init value directly into the eval function (eval uses the init-time value; no runtime slot needed) and treat an Invalid mapping like the existing dead-value path (default to 0). Only a genuine instruction result takes the slot+optbarrier path, so no cache slot or codegen ever sees a constant/undefined value again. (2) hir_ty/inference.rs -- ddx(V(p,n), 5) (a ddx whose 2nd arg is not a potential/flow probe) crashed hir_lower at value_def(unknown).unwrap_param(). The type-checker HAD an "invalid ddx unknown" diagnostic, but its guard tested expr (the ddx call itself, ALWAYS an Expr::Call) instead of unknown, so the diagnostic was DEAD CODE and the malformed call slipped through to codegen. FIX: test unknown -- a non-call unknown (a literal, a plain variable) now raises the diagnostic and compilation stops before lowering. (This also newly (correctly) diagnoses the ddx(1.0,1.0)/ddx(1.0,foo) "random fuzz" cases the ui/ddx.va test already carried but that the dead code never rejected -- ddx.log updated to 8 errors.) (3) hir/elaborate.rs -- a malformed top-level module whose item TREE recorded an instantiation but whose parsed AST module_items() came back empty (parser error-recovery and item-tree construction disagreeing) crashed the E-5/E-49 hierarchy-flattening pass at items.first().unwrap(). FIX: an empty AST item list means nothing to flatten -- return the module text verbatim, the same no-op the pass already takes for a module with no instantiations. Behaviour-preserving for valid input: full dual-solver regression 214/214, cargo test unchanged except the intended ddx.log update (no MIR/OSDI snapshot changed -- no real model exercises the new paths), and a re-fuzz of the fixed compiler across all three strategies finds no surviving panics. Verify (examples/vafcrash4): a minimal reproducer per cause now compiles cleanly (nested analog ops) or clean-errors (ddx, malformed module), each having exited 101 on the shipped binary. Three source files + one snapshot.

E-264 *hir (elaborate.rs), osdi (lib.rs), examples/vafhang_examples/**

large instance arrays: hierarchy-flatten O(N²)->O(N) + OSDI-codegen stack headroom for deep per-node fan-in. Two scalability/robustness defects on the same path -- compiling a module that instantiates a large array of a sub-module. (1) FLATTEN O(N²)->O(N) (hir/elaborate.rs): the E-5/E-49/E-86 pass renders one textual copy of a sub-module per instance with a per-instance name prefix and rewrites hierarchical name references, running N times for leaf u[0:N]. Three inner steps were themselves O(N), making it O(N²) -- doubling N QUADRUPLED time (2k≈1.

8s, 8k≈30s, 16k≈100s), so a big array looked like a hang: hierarchical-name resolution (`find_instance_path_holes`) re-scanned every instance prefix per token via `.keys().any(..)`; it was called per port binding, per instance (multiplying by N again); and each per-instance scope `clone()`d the whole E-86 absolute-reference map (O(N) entries). FIX (behaviour-preserving): precompute an ANCESTOR SET once for O(1) ”is this chain a hierarchical prefix?” tests (no per-token scan); a DOT-FREE EARLY-OUT (every hierarchical ref contains a `.`, so the common no-dot token skips resolution entirely); and share the absolute-reference map by reference -- a small `Rc<AbsPrefixes>` (map + its precomputed ancestor set) is `Rc::cloned` (O(1)) into each scope instead of deep-copied. Now O(N): 16 001 instances elaborate+compile in ~1s (was ~100s), 32 001 in ~2s; the elaborated text and every resulting `.osdi` are IDENTICAL (only faster). (2) CODEGEN STACK (`osdi/lib.rs`): when many instances contribute to the SAME node and don't collapse (distinct per-instance parameter), the residual is a chain of thousands of accumulated contributions that OpenVAF's recursive OSDI codegen + autodiff walk; that recursion ran on a rayon codegen worker whose default stack is only a few MB, so past ~8 000 contributions it aborted with `thread ... has overflowed its stack` / SIGABRT -- a crash on large-but-valid input. FIX: run OSDI codegen on a dedicated rayon pool built with a 256 MiB worker stack instead of the default global pool. A thread's stack size is a property of the thread, not the work -- it cannot change the emitted code (the `.osdi` is byte-equivalent modulo the linker's already-nondeterministic build id) -- it only lets the deep recursion complete; the 8 001-contribution design now compiles cleanly. Behaviour-preserving: full dual-solver regression passes and `cargo test` is unchanged (no MIR/OSDI snapshot moved -- the flatten fix only accelerates identical work; the stack fix is a thread property). Extreme instance counts remain bounded by the existing 1 M array/generate caps and downstream compile cost. Verify (examples/vafhang): check A ( always) -- 16 001- and 32 001-instance arrays compile in ~1s/~2s within generous absolute bounds a re-introduced O($N^2$) would blow; check B (`--slow`) -- the 8 001-contribution deep-fan-in design compiles cleanly (~38s) where the pre-fix binary SIGABRT'd. Two source files.

Enhancements not listed (57, 60, 62–64, 69, 72–77, 79–83, 94–95, 98–100) changed no compiler sources — they were validation suites, documentation, benchmark tooling, or ngspice-side work (see the ngspice report).